

Securing Audit Logs with Hash Chains and HMAC

Eksamen i Secure Software Development

Projektgruppe: [Mikkel Bak Markers]

Dato: 29. maj 2026

Indhold

1	Introduktion	3
1.1	Projektets Formål	3
1.2	Afgrænsning	3
2	System-Arkitektur	4
2.1	Overordnet Design	4
2.2	Komponent Diagram	4
2.3	Komponenter	4
2.3.1	AuditLog (Kerne)	4
2.3.2	KeyManager	4
2.3.3	LogVerifier	4
2.3.4	JsonLogStorage	5
3	Datamodeller	6
3.1	Primære Datastrukturer	6
3.1.1	LogEntry	6
3.1.2	KeyVersion	6
3.1.3	VerificationReport	6
4	Implementering	8
4.1	Teknologivalg	8
4.2	Centrale Funktioner	8
4.2.1	Entry Appending	8
4.2.2	Integrity Verification	8
4.2.3	Key Rotation	9
4.3	Lagring og persistence	9
5	Verifikation og Test	10
5.1	Test-Strategi	10
5.2	Demonstrativ Test	10
6	Kildekode-Oversigt	11
6.1	Vigtige Filer	11
6.2	Mapestruktur	11
6.3	Designmønstre	12
7	Konklusion	13
7.1	Resultater	13
7.2	Sikkerhedsovervejelser	13

7.2.1	Styrker	13
7.2.2	Begrænsninger	13
7.3	Afsluttende Bemærkninger	13

1 Introduktion

1.1 Projektets Formål

Mit mål med opgaven er, at demonstrere hvordan man kan implementere sikkerhedsforanstaltninger i et system der håndterer ”audit logs”. Fokus ligger på at sikre integritet af logs ved hjælp af hash-kæder og HMAC. Samtidigt vil jeg belyse de udfordringer og overvejelser der er forbundet med at implementere sådanne mekanismer i praksis.

1.2 Afgrænsning

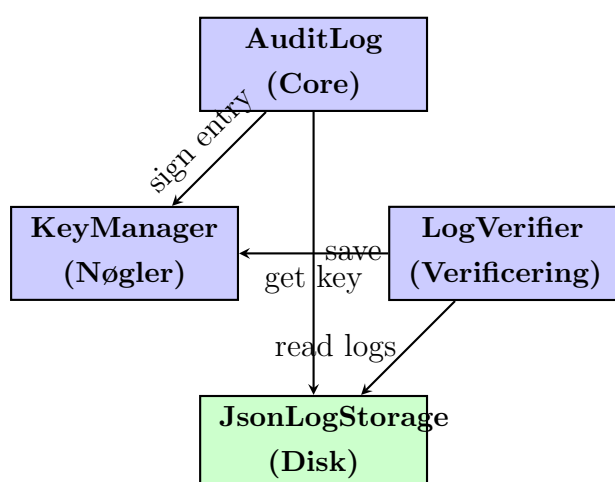
Projektet fokuserer udelukkende på implementeringer af kryptografiske tricks for at sikre integriteten af audit logs. Det vil ikke dække andre aspekter af sikkerhed, hverken adgangskontrol eller databeskyttelse.

2 System-Arkitektur

2.1 Overordnet Design

Systemet består af fire hovedkomponenter som arbejder sammen for at sikre integritet og autentificitet af audit logs. Arkitekturen er bygget omkring princippet om append-only logs med kryptografisk binding mellem successive entries.

2.2 Komponent Diagram



Data flow: AuditLog koordinerer signing og storage, LogVerifier verificerer integritet mod logs i storage.

2.3 Komponenter

2.3.1 AuditLog (Kerne)

Hovedklassen som håndterer oprettelse af nye log entries. Den koordinerer med KeyManager for at sign entries og sender dem til JsonLogStorage for persistence. Hver entry linkes til den forrige gennem hash-værdier.

2.3.2 KeyManager

Administrerer kryptografiske nøgler og deres versioner. Tillader nøglerotation uden at ødelægge verificering af ældre entries. Holder styr på hvilken nøgleversion hver entry blev signed med.

2.3.3 LogVerifier

Verificerer integriteten af hele log-kæden. Gennemgår hver entry og kontrollerer:

- At hash-kæden er intakt (prevHash matcher forrige entry)

- At HMAC er gyldig med den korrekte nøgleversion
- At ingen entries er slettet eller manipuleret

2.3.4 JsonLogStorage

Håndterer persistence af log entries til disk i JSON-format. Implementerer append-only principper og giver læseadgang til hele historien.

3 Datamodeller

3.1 Primære Datastrukturer

Systemet arbejder med tre centrale datastrukturer: `LogEntry`, `KeyVersion`, og `VerificationReport`.

3.1.1 `LogEntry`

En `LogEntry` repræsenterer en enkelt hændelse i audit log-kæden:

- **Timestamp**: UTC-tid for når hændelsen blev registreret
- **Event**: Streng der beskriver typen af hændelse (f.eks. "USER_LOGIN", "FILE_MODIFIED")
- **Data**: Vilkårlig data forbundet med hændelsen
- **PrevHash**: SHA256-hash af den forrige entry - danner kæden
- **Hash**: SHA256-hash af denne entry's indhold
- **Hmac**: HMAC-SHA256 signatur for autenticitet
- **KeyVersion**: Hvilken nøgleversion blev brugt til at sign denne entry

3.1.2 `KeyVersion`

Repræsenterer en kryptografisk nøgle og dens metadata:

- **Version**: Numerisk identifikator (1, 2, 3, osv.)
- **SecretKey**: Den faktiske hemmelighed brugt til HMAC
- **CreatedAt**: Hvornår nøglen blev oprettet
- **IsActive**: Om dette er den nuværende nøgle for nye entries

3.1.3 `VerificationReport`

Detaljeret rapport fra verificering af log-integritet:

- **IsValid**: Boolean - hele kæden gyldig?
- **TotalEntries**: Antal entries der blev kontrolleret
- **ValidEntries**: Hvor mange var gyldige
- **InvalidEntries**: Hvor mange fejlede
- **FirstInvalidEntryIndex**: Indeks på første fejl (hvis nogen)

- **ErrorMessage:** Beskrivelse af hvad der gik galt
- **VerificationTime:** Hvornår verificeringen blev udført

4 Implementering

4.1 Teknologivalg

Projektet er implementeret i **C# (.NET 9)** med følgende begrundelser:

- **System.Security.Cryptography:** Built-in kryptografisk bibliotek med SHA256 og HMAC-SHA256
- **JSON-serialisering:** Læsbar persistence af entries til disk
- **Stærk typesikkerhed:** C#'s typesystem opdager fejl ved compile-time
- **Exception handling:** Præcis fejlhåndtering gennem hele systemet

4.2 Centrale Funktioner

4.2.1 Entry Appending

`AuditLog.Append(event, data)` tilføjer en ny entry:

1. Hent hash fra sidste entry (eller "0" hvis tom log)
2. Generer nuværende timestamp
3. Opbyg entry-indhold fra timestamp, event, data, prevHash
4. Beregn SHA256 af indhold
5. Beregn HMAC med aktiv nøgle
6. Opret LogEntry-objekt
7. Gem til disk via storage-interface

4.2.2 Integrity Verification

`LogVerifier.VerifyWithReport(entries)` verificerer hele kæden og rapporterer problemer:

1. For hver entry: check at `prevHash` matcher forrige entries `hash`
2. Hent den korrekte nøgleversion baseret på entries `keyVersion`
3. Genberegnet HMAC - skal matche lagret værdi
4. Genberegnet SHA256-hash - skal matche lagret værdi
5. Hvis fejl findes: rapportér indeks og årsag
6. Returnér detaljeret rapport med statistik

4.2.3 Key Rotation

`KeyManager.RotateKey()` introducerer ny nøgle:

1. Generer ny hemmelighed med `RandomNumberGenerator`
2. Gem ny nøgle med version nummer og timestamp
3. Markér som aktiv
4. Gamle nøgler gemmes for at verificere historiske entries

4.3 Lagring og persistence

`JsonLogStorage` håndterer persistence:

- **Append-only:** Nye entries føjes til slutningen af JSON-fil
- **JSON-format:** Human-readable struktur gør manuel inspektion muligt
- **Atomare operationer:** Hele lister skrives på én gang for consistency
- **Deserialisering:** Læs entries fra disk og rekonstruer objekter

Eksempel på lagret JSON-struktur:

```
[
  {
    "timestamp": "2026-05-29T22:30:00Z",
    "event": "USER_LOGIN",
    "data": "Admin login",
    "prevHash": "0",
    "hash": "a3f2e1c9...",
    "hmac": "c5e9d1a2...",
    "keyVersion": 1
  }
]
```

5 Verifikation og Test

5.1 Test-Strategi

Systemet verificeres gennem tre lag:

1. **Hash-kæde integritet:** Hver entry skal linke korrekt til den forrige
2. **HMAC autentificering:** Hver entry skal have gyldig signatur fra korrekt nøgle
3. **Hash konsistens:** Genberegnet SHA256 skal matche lagret værdi

Hvis nogle af disse fejler, rapporteres:

- Præcis hvilken entry fejlede
- Hvilken check der fejlede (prevHash, HMAC, eller hash)
- Hvornår fejlen blev detekteret
- Samlet statistik over gyldige og ugyldige entries

5.2 Demonstrativ Test

Demo-program (`Program.cs`) udfører følgende scenarier:

1. **Opret initial log:** Flere normale entries tilføjes
2. **Verificer initial log:** Alle entries skal være gyldige
3. **Nøglerotation:** Ny nøgle introduceres
4. **Tilføj entries med ny nøgle:** Entries bliver signed med ny nøgle
5. **Verificer igen:** Hele kæde med blandet nøgleversioner skal være valid
6. **Tampering-test:** Manipulér en entry og vis at verificering fejler
7. **Rapportér resultater:** Detaljerede rapporter vises for hver test

Disse tests demonstrerer systemets evne til at:

- Bevare integritet over lange perioder
- Håndtere nøglerotation uden at miste verificering
- Detektere enhver manipulation med præcision

6 Kildekode-Oversigt

6.1 Vigtige Filer

- `AuditLogSystem/Core/LogEntry.cs` - Datamodel for en log entry
- `AuditLogSystem/Core/AuditLog.cs` - Hovedklasse for log-håndtering
- `AuditLogSystem/Core/KeyManager.cs` - Nøgle- og versionshåndtering
- `AuditLogSystem/Core/CryptoUtil.cs` - Kryptografiske hjælpefunktioner (SHA256, HMAC)
- `AuditLogSystem/Verification/LogVerifier.cs` - Verificering af log-integritet
- `AuditLogSystem/Verification/VerificationReport.cs` - Detaljeret verificeringsrapport
- `AuditLogSystem/Storage/JsonLogStorage.cs` - JSON-baseret persistence
- `Program.cs` - Demo-program der viser systemet i aktion

6.2 Mappestuktur

SSD_EXAM/

AuditLogSystem/

Core/

AuditLog.cs

AuditLogException.cs

CryptoUtil.cs

KeyManager.cs

KeyVersion.cs

LogEntry.cs

Storage/

ILogStorage.cs

JsonLogStorage.cs

Verification/

LogVerifier.cs

VerificationReport.cs

Report/

SSD_EXAM_REPORT.tex

Program.cs

SSD_EXAM.csproj

audit_log.json

6.3 Designmønstre

Dependency Injection: `AuditLog` modtager `KeyManager` og `ILogStorage` via konstruktor, gør systemet modulært og testbart.

Strategy Pattern: `ILogStorage` interface tillader forskellige lagringsimplementeringer uden at ændre kernalogin.

Immutable DataModels: `LogEntry` har kun read-only properties, forhindrer uhørt modificering af historiske data.

7 Konklusion

Dette projekt har demonstreret, hvordan man kan implementere et robust og tamper-evident audit log system ved hjælp af relativt simple kryptografiske primitiver.

7.1 Resultater

Systemet opfylder sine kernekrav:

- **Integritet:** Hash-kæder sikrer, at enhver ændring af historiske data detekteres
- **Autentificitet:** HMAC-signaturer beviser, at entries blev oprettet af systemet selv
- **Nøglerotation:** Gamle entries forbliver verificable, selvom nøgler roteres
- **Transparens:** JSON-format gør det muligt at inspicere logs uden særlige værktøjer

7.2 Sikkerhedsovervejelser

7.2.1 Styrker

- Hash-kæder og HMAC er matematisk vellykkede mekanismer
- Systemet detekterer tampering, sletning og forfalskelse
- Nøglerotation uden at miste verificering af historiske data

7.2.2 Begrænsninger

- Hvis krypteringsnøglen bliver kompromitteret, kan nye entries forges
- Systemet beskytter ikke mod påtvunget sletning af hele log-filen
- Ingen redundans eller backup-mekanismer implementeret
- Kræver sikker nøglestorage (uden for dette projekts scope)

7.3 Afsluttende Bemærkninger

Projektet viser, at “Security by Design” ikke nødvendigvis kræver kompleks kryptografi. Ved at kombinere velkendte primitiver (SHA256, HMAC) med god arkitektur (append-only logs, nøgleversioner) kan man opnå stærke sikkerhedsgarantier for audit logs.

I en produktionsmiljø ville man skulle tilføje:

- Sikker lagring af krypteringsnøgler (hardware security modules, key vaults)
- Distribueret redundans af logs

- Adgangskontrol til log-data
- Compliance med GDPR, CRA og anden lovgivning

Denne implementation tjener som et solidt fundament for disse yderligere forbedringer.